secureverse

ccpa - Compliance report

Sun Mar 08 2026 18:11:41 GMT+0000 (Coordinated Universal Time)

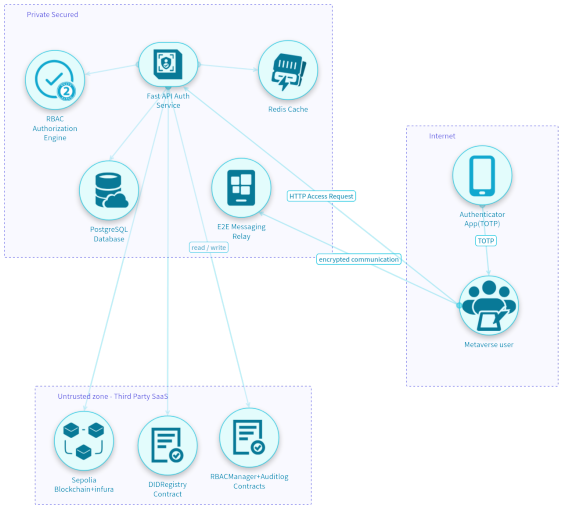
Project description: *No description*

Unique ID: secureverse-1772967171562

Owner: [Isra Chaudry]

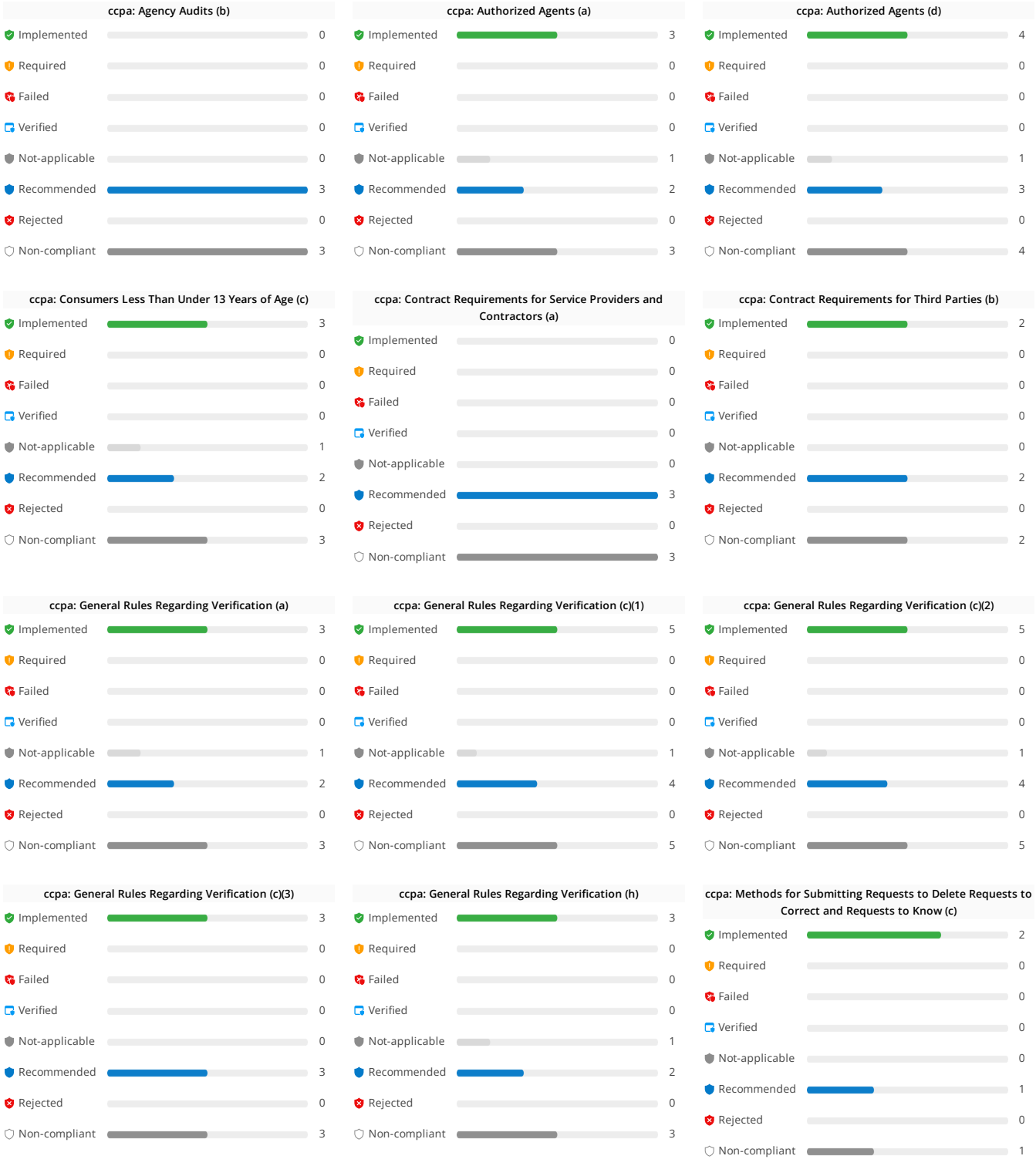
Workflow state: Final Version

Tags: *No tags*



Compliance summary

See the countermeasure state data categorized by standards:



ccpa: Methods for Submitting Requests to Delete Requests to Correct and Requests to Know (d)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Notice at Collection of Personal Information (a)(b)(c)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Notice at Collection of Personal Information (g)(1)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Notice at Collection of Personal Information (g)(2)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Opt-Out Preference Signals (c)(2)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	1
Recommended	5
Rejected	0
Non-compliant	6

ccpa: Opt-Out Preference Signals (c)(5)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Opt-Out Preference Signals (c)(6)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Opt-Out Preference Signals (d)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Opt-Out Preference Signals (g)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	1
Recommended	5
Rejected	0
Non-compliant	6

ccpa: Personal Information Security Breaches (a)(1)(2)

Implemented	6
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	11
Rejected	0
Non-compliant	11

ccpa: Privacy Policy (e)(2)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Privacy Policy (e)(3)(E)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Privacy Policy (e)(3)(G)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Privacy Policy (e)(1)(I)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Record-Keeping (a)

Implemented	0
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	3
Rejected	0
Non-compliant	3

ccpa: Record-Keeping (b)

Implemented	0
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Record-Keeping (d)

Implemented	1
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Correct (a)

Implemented	3
Required	0
Failed	0
Verified	0
Not-applicable	1
Recommended	2
Rejected	0
Non-compliant	3

ccpa: Requests to Correct (k)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Delete (b)(3)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Know (a)

Implemented	3
Required	0
Failed	0
Verified	0
Not-applicable	1
Recommended	2
Rejected	0
Non-compliant	3

ccpa: Requests to Know (b)

Implemented	3
Required	0
Failed	0
Verified	0
Not-applicable	1
Recommended	2
Rejected	0
Non-compliant	3

ccpa: Requests to Know (k)(1)

Implemented	1
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (a)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(2)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(3)

Implemented	0
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(8)

Implemented	1
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Opt-Out of Sale/Sharing (a)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Opt-Out of Sale/Sharing (d)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Opt-Out of Sale/Sharing (f)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Opt-Out of Sale/Sharing (g)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Opt-Out of Sale/Sharing (h)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Opt-Out of Sale/Sharing (j)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requests to Opt-Out of Sale/Sharing (k)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Requirements for Businesses Collecting Large Amounts of Personal Information (a)(1)

Implemented	1
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Restrictions on the Collection and Use of Personal Information (b)(5)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Restrictions on the Collection and Use of Personal Information (c)(2)

Implemented	1
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Restrictions on the Collection and Use of Personal Information (c)(3)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Restrictions on the Collection and Use of Personal Information (f)

Implemented	1
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Service Providers and Contractors (a)(5)

Implemented	2
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Sworn Complaints Filed with the Agency (a)(b)

Implemented	0
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Third Parties (b)

Implemented	1
Required	0
Failed	0
Verified	0
Not-applicable	0
Recommended	1
Rejected	0
Non-compliant	1

ccpa: Verification for Non-Accountholders (b)

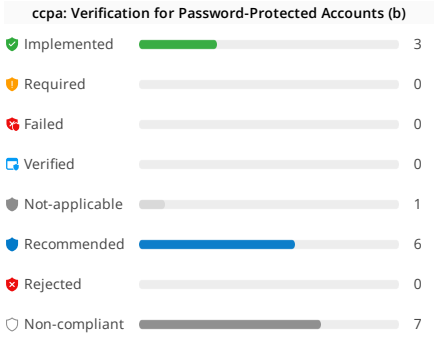
Implemented	3
Required	0
Failed	0
Verified	0
Not-applicable	1
Recommended	2
Rejected	0
Non-compliant	3

ccpa: Verification for Non-Accountholders (c)

Implemented	3
Required	0
Failed	0
Verified	0
Not-applicable	1
Recommended	2
Rejected	0
Non-compliant	3

ccpa: Verification for Password-Protected Accounts (a)

Implemented	4
Required	0
Failed	0
Verified	0
Not-applicable	2
Recommended	7
Rejected	0
Non-compliant	9


ccpa: Verification for Password-Protected Accounts (b)
Implemented 3
Required 0
Failed 0
Verified 0
Not-applicable 1
Recommended 6
Rejected 0
Non-compliant 7

Content menu

ccpa: Authorized Agents (a)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Authorized Agents (d)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Agency Audits (b)
- Non-compliant
- Recommended

ccpa: Contract Requirements for Service Providers and Contractors (a)
- Non-compliant
- Recommended

ccpa: Consumers Less Than Under 13 Years of Age (c)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Contract Requirements for Third Parties (b)
- Implemented
- Non-compliant
- Recommended

ccpa: General Rules Regarding Verification (a)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: General Rules Regarding Verification (c)(1)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: General Rules Regarding Verification (c)(2)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: General Rules Regarding Verification (c)(3)
- Implemented
- Non-compliant
- Recommended

ccpa: General Rules Regarding Verification (h)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Personal Information Security Breaches (a)(1)(2)
- Implemented
- Non-compliant
- Recommended

ccpa: Requests to Correct (a)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Requests to Know (a)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Requests to Know (b)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(8)
- Implemented
- Non-compliant
- Recommended

ccpa: Requirements for Businesses Collecting Large Amounts of Personal Information (a)(1)
- Implemented
- Non-compliant
- Recommended

ccpa: Third Parties (b)
- Implemented
- Non-compliant
- Recommended

ccpa: Verification for Non-Accountholders (b)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Verification for Non-Accountholders (c)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Verification for Password-Protected Accounts (a)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Opt-Out Preference Signals (c)(2)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Opt-Out Preference Signals (g)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Verification for Password-Protected Accounts (b)
- NotApplicable
- Implemented
- Non-compliant
- Recommended

ccpa: Requests to Know (k)(1)
- Implemented
- Non-compliant
- Recommended

ccpa: Restrictions on the Collection and Use of Personal Information (c)(2)
- Implemented
- Non-compliant
- Recommended

ccpa: Restrictions on the Collection and Use of Personal Information (f)
- Implemented
- Non-compliant
- Recommended

ccpa: Record-Keeping (a)
- Non-compliant
- Recommended

ccpa: Methods for Submitting Requests to Delete Requests to Correct and Requests to Know (c)
- Implemented

Non-compliant

Recommended

ccpa: Methods for Submitting Requests to Delete Requests to Correct and Requests to Know (d)

Implemented

Non-compliant

Recommended

ccpa: Notice at Collection of Personal Information (a)(b)(c)

Implemented

Non-compliant

Recommended

ccpa: Notice at Collection of Personal Information (g)(1)

Implemented

Non-compliant

Recommended

ccpa: Notice at Collection of Personal Information (g)(2)

Implemented

Non-compliant

Recommended

ccpa: Opt-Out Preference Signals (c)(6)

Implemented

Non-compliant

Recommended

ccpa: Opt-Out Preference Signals (d)

Implemented

Non-compliant

Recommended

ccpa: Privacy Policy (e)(2)

Implemented

Non-compliant

Recommended

ccpa: Privacy Policy (e)(3)(E)

Implemented

Non-compliant

Recommended

ccpa: Privacy Policy (e)(3)(G)

Implemented

Non-compliant

Recommended

ccpa: Privacy Policy (e)(1)(I)

Implemented

Non-compliant

Recommended

ccpa: Requests to Correct (k)

Implemented

Non-compliant

Recommended

ccpa: Requests to Delete (b)(3)

Implemented

Non-compliant

Recommended

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (a)

Implemented

Non-compliant

Recommended

ccpa: Requests to Opt-Out of Sale/Sharing (a)

Implemented

Non-compliant

Recommended

ccpa: Requests to Opt-Out of Sale/Sharing (d)

Implemented

Non-compliant

Recommended

ccpa: Requests to Opt-Out of Sale/Sharing (f)

Implemented

Non-compliant
Recommended

ccpa: Requests to Opt-Out of Sale/Sharing (g)
Implemented
Non-compliant
Recommended

ccpa: Requests to Opt-Out of Sale/Sharing (h)
Implemented
Non-compliant
Recommended

ccpa: Requests to Opt-Out of Sale/Sharing (j)
Implemented
Non-compliant
Recommended

ccpa: Requests to Opt-Out of Sale/Sharing (k)
Implemented
Non-compliant
Recommended

ccpa: Restrictions on the Collection and Use of Personal Information (b)(5)
Implemented
Non-compliant
Recommended

ccpa: Restrictions on the Collection and Use of Personal Information (c)(3)
Implemented
Non-compliant
Recommended

ccpa: Service Providers and Contractors (a)(5)
Implemented
Non-compliant
Recommended

ccpa: Opt-Out Preference Signals (c)(5)
Implemented
Non-compliant
Recommended

ccpa: Record-Keeping (d)
Implemented
Non-compliant
Recommended

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(2)
Implemented
Non-compliant
Recommended

ccpa: Record-Keeping (b)
Non-compliant
Recommended

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(3)
Non-compliant
Recommended

ccpa: Sworn Complaints Filed with the Agency (a)(b)
Non-compliant
Recommended

ccpa: Authorized Agents (a)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Authorized Agents (d)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low
Imp2. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Agency Audits (b)

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

3 Recommended countermeasures

Rec1. Configure automatic mobile OS updates C-MOBILE-DEVICE-CLIENT-CNT-02 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update automatic mobile OS updates on all mobile devices to ensure they are running the latest secure versions of their operating systems. This control minimizes exposure to vulnerabilities by promptly applying critical patches and security improvements as they are released. Developers and DevOps engineers should configure mobile device management (MDM) solutions or native OS settings to enforce automatic updates and continuously monitor update compliance across the device fleet.
  Implementation Steps:
  Enable Auto-Updates:
  Configure mobile devices to automatically download and install OS updates as soon as they are available.
  Deploy Centralized Update Policies:
  Use MDM solutions to enforce and manage update policies across all mobile devices, ensuring uniform compliance.
  Monitor Update Compliance:
  Regularly audit update logs and compliance reports to verify that all devices are running the latest OS versions and apply remediation if discrepancies are found.
  References:
  - Android Enterprise Updates

Rec2. Deploy mobile anti-malware solutions C-MOBILE-DEVICE-CLIENT-CNT-04 Very high Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile anti-malware solutions on all mobile devices to detect, prevent, and remediate malware infections that could compromise the device's security. This control ensures that reputable anti-malware software with real-time scanning and periodic full-device scans is installed and maintained. Developers and DevOps engineers should use mobile device management (MDM) tools to enforce consistent deployment, configuration, and automatic updates of malware definitions across the entire mobile device fleet.
  Implementation Steps:
  Install Reputable Anti-Malware Software:
  Deploy trusted anti-malware applications on all mobile devices via MDM solutions or approved installation packages.
  Enable Real-Time Scanning:
  Configure the anti-malware solution to continuously monitor device activity and block potential threats in real time.
  Schedule Regular Full-Device Scans:
  Set up periodic scans to identify and remediate any dormant or emerging malware infections.
  Automate Malware Definition Updates:
  Ensure the anti-malware software is configured to automatically update its threat definitions to keep pace with emerging threats.
  Monitor and Audit Compliance:
  Use centralized management tools to verify that all devices maintain up-to-date anti-malware protection and address any compliance issues promptly.
  References:
  - NIST SP 800-83: Guide to Malware Incident Prevention and Handling

Rec3. Deploy mobile anti-phishing protection C-MOBILE-DEVICE-CLIENT-CNT-06 Very high Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update anti-phishing protection on all mobile devices to detect and block phishing attempts targeting mobile users. This control leverages built-in email and browser security features, integrates threat intelligence feeds, and enforces centralized policies to ensure that phishing attacks are promptly identified and mitigated. Developers and DevOps engineers should use mobile device management (MDM) solutions to deploy and monitor anti-phishing configurations consistently across the entire mobile device fleet.
  Implementation Steps:
  Enable Built-In Anti-Phishing Features:
  Activate anti-phishing functionalities available in mobile email and browser applications, ensuring they are properly configured to warn users of suspicious links and communications.
  Integrate Threat Intelligence Feeds:
  Connect to reputable threat intelligence services (e.g., Google Safe Browsing) to keep phishing detection databases current with emerging phishing trends.
  Deploy Centralized Policies:
  Use MDM tools to enforce uniform anti-phishing configurations and update policies across all mobile devices, ensuring consistent protection.
  Conduct Regular Testing:
  Periodically simulate phishing attacks and audit mobile device logs to verify the effectiveness of anti-phishing measures and adjust configurations as needed.
  References:
  - Google Safe Browsing API

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Configure automatic mobile OS updates C-MOBILE-DEVICE-CLIENT-CNT-02 High
Rec2. Deploy mobile anti-malware solutions C-MOBILE-DEVICE-CLIENT-CNT-04 Very high
Rec3. Deploy mobile anti-phishing protection C-MOBILE-DEVICE-CLIENT-CNT-06 Very high

ccpa: Contract Requirements for Service Providers and Contractors (a)

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

3 Recommended countermeasures

Rec1. Configure automatic mobile OS updates C-MOBILE-DEVICE-CLIENT-CNT-02 High Not tested

- State: Recommended
- Description:
  Implement and regularly update automatic mobile OS updates on all mobile devices to ensure they are running the latest secure versions of their operating systems. This control minimizes exposure to vulnerabilities by promptly applying critical patches and security improvements as they are released. Developers and DevOps engineers should configure mobile device management (MDM) solutions or native OS settings to enforce automatic updates and continuously monitor update compliance across the device fleet.
  Implementation Steps:
  Enable Auto-Updates:
  Configure mobile devices to automatically download and install OS updates as soon as they are available.
  Deploy Centralized Update Policies:
  Use MDM solutions to enforce and manage update policies across all mobile devices, ensuring uniform compliance.
  Monitor Update Compliance:
  Regularly audit update logs and compliance reports to verify that all devices are running the latest OS versions and apply remediation if discrepancies are found.
  References:
  - Android Enterprise Updates

Rec2. Deploy mobile anti-malware solutions C-MOBILE-DEVICE-CLIENT-CNT-04 Very high Not tested

- State: Recommended
- Description:
  Implement and regularly update mobile anti-malware solutions on all mobile devices to detect, prevent, and remediate malware infections that could compromise the device's security. This control ensures that reputable anti-malware software with real-time scanning and periodic full-device scans is installed and maintained. Developers and DevOps engineers should use mobile device management (MDM) tools to enforce consistent deployment, configuration, and automatic updates of malware definitions across the entire mobile device fleet.
  Implementation Steps:
  Install Reputable Anti-Malware Software:
  Deploy trusted anti-malware applications on all mobile devices via MDM solutions or approved installation packages.
  Enable Real-Time Scanning:
  Configure the anti-malware solution to continuously monitor device activity and block potential threats in real time.
  Schedule Regular Full-Device Scans:
  Set up periodic scans to identify and remediate any dormant or emerging malware infections.
  Automate Malware Definition Updates:
  Ensure the anti-malware software is configured to automatically update its threat definitions to keep pace with emerging threats.
  Monitor and Audit Compliance:
  Use centralized management tools to verify that all devices maintain up-to-date anti-malware protection and address any compliance issues promptly.
  References:
  - NIST SP 800-83: Guide to Malware Incident Prevention and Handling

Rec3. Deploy mobile anti-phishing protection C-MOBILE-DEVICE-CLIENT-CNT-06 Very high Not tested

- State: Recommended
- Description:
  Implement and regularly update anti-phishing protection on all mobile devices to detect and block phishing attempts targeting mobile users. This control leverages built-in email and browser security features, integrates threat intelligence feeds, and enforces centralized policies to ensure that phishing attacks are promptly identified and mitigated. Developers and DevOps engineers should use mobile device management (MDM) solutions to deploy and monitor anti-phishing configurations consistently across the entire mobile device fleet.
  Implementation Steps:
  Enable Built-In Anti-Phishing Features:
  Activate anti-phishing functionalities available in mobile email and browser applications, ensuring they are properly configured to warn users of suspicious links and communications.
  Integrate Threat Intelligence Feeds:
  Connect to reputable threat intelligence services (e.g., Google Safe Browsing) to keep phishing detection databases current with emerging phishing trends.
  Deploy Centralized Policies:
  Use MDM tools to enforce uniform anti-phishing configurations and update policies across all mobile devices, ensuring consistent protection.
  Conduct Regular Testing:
  Periodically simulate phishing attacks and audit mobile device logs to verify the effectiveness of anti-phishing measures and adjust configurations as needed.
  References:
  - Google Safe Browsing API

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Configure automatic mobile OS updates C-MOBILE-DEVICE-CLIENT-CNT-02 High
Rec2. Deploy mobile anti-malware solutions C-MOBILE-DEVICE-CLIENT-CNT-04 Very high
Rec3. Deploy mobile anti-phishing protection C-MOBILE-DEVICE-CLIENT-CNT-06 Very high

ccpa: Consumers Less Than Under 13 Years of Age (c)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Contract Requirements for Third Parties (b)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Component: RBACManager+Auditlog Contracts

Imp1. Design proper access controls for Smart Contract C-SMART-CONTRACT-CNT-01 Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile

device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
Implementation Steps:
Enable Auto-Updates for Mobile Apps:
Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
Deploy Vulnerability Scanning Tools:
Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
Integrate with Mobile Device Management (MDM):
Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
Monitor, Report, and Remediate:
Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
References:
- OWASP Mobile Security Testing Guide

Component: DIDRegistry Contract

1 Recommended countermeasures

Rec1. Design proper access controls for Smart Contract C-SMART-CONTRACT-CNT-01 High Not tested

- **State:** Recommended
- **Description:**
To bolster the security of smart contracts, it's crucial to establish robust access controls. In the absence of adequate access controls, unauthorized entities can carry out sensitive operations, potentially exposing significant vulnerabilities. This approach provides an overview of several access control strategies, such as the Ownable pattern, role-based access control, and multi-signature wallets, to deter unauthorized utilization of smart contract functionalities.
Remediation:
1. Ownable Pattern
- During contract creation, designate an address as the "owner" of the contract.
- For protected functions, apply the "OnlyOwner" modifier. This modifier ensures that the contract authenticates the calling address's identity before executing the function.
- When calls to protected functions originate from addresses other than the contract owner, they should revert, denying unauthorized access.
2. Role-Based Access Control
- Distribute access to sensitive functions among a set of trusted participants, each assigned specific administrative roles. For example, one account may be responsible for minting tokens, while another handles contract upgrades or pausing.
- Implement modifiers or conditional checks within functions to ensure that only authorized roles can execute them.
- Role-based access control mitigates centralization risks associated with a single owner. Even if one account's keys are compromised, it doesn't jeopardize the entire contract.
3. Using Multi-Signature Wallets
- Utilize a multi-signature account, which requires signatures from multiple entities to execute transactions, typically a minimum number of accounts out of a predefined group.
- Configure the multi-signature wallet to require consensus from a specified number of account holders before executing actions on the target contract.
- Multi-signature wallets add an additional layer of security. This approach is particularly useful when implementing the Ownable pattern, as it complicates unauthorized manipulation of contract functions.
References:
- Smart contract security

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: DIDRegistry Contract

Rec1. Design proper access controls for Smart Contract C-SMART-CONTRACT-CNT-01 High

ccpa: General Rules Regarding Verification (a)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: General Rules Regarding Verification (c)(1)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low
Imp2. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: PostgreSQL Database

Imp1. Secure setup and access control C-DBAAS-DATABASE-AS-A-SERVICE-SECURE-SETUP Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience.
  Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

2 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side.
  Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High
Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: General Rules Regarding Verification (c)(2)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low
Imp2. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: PostgreSQL Database

Imp1. Secure setup and access control C-DBAAS-DATABASE-AS-A-SERVICE-SECURE-SETUP Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

2 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side. Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.

2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High
Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: General Rules Regarding Verification (c)(3)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: PostgreSQL Database

Imp1. Secure setup and access control C-DBAAS-DATABASE-AS-A-SERVICE-SECURE-SETUP Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience.
  Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**

Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
Implementation Steps:
Enable Auto-Updates for Mobile Apps:
Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
Deploy Vulnerability Scanning Tools:
Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
Integrate with Mobile Device Management (MDM):
Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
Monitor, Report, and Remediate:
Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
References:
- OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side. Follow these steps:

1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High

ccpa: General Rules Regarding Verification (h)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.

Implementation Steps:
Enable Auto-Updates for Mobile Apps:
Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
Deploy Vulnerability Scanning Tools:
Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
Integrate with Mobile Device Management (MDM):
Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
Monitor, Report, and Remediate:
Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
References:
- OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Personal Information Security Breaches (a)(1)(2)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: Fast API Auth Service

Imp1. Use secure access control mechanisms C-COMPREHENSIVE-SECURITY-MEASURES-FOR-ACCESS-CONTROL-AND-CONFIGURATION Low

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low
Imp2. Enforce principle of least privilege in scope management C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-06 Low

Component: E2E Messaging Relay

Imp1. Use secure storage mechanisms C-SECURE-STORAGE Low

Component: Metaverse user

Imp1. Consider defenses against adversarial tampering such as adversarial training methods or data preprocessing-based approaches C-ML-AI-ADVERSARIAL-ATTACKS High
Imp2. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

2 Recommended countermeasures

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High Not tested

- **State:** Recommended
- **Description:**
Implement secure configuration and encryption standards to protect Authentication and Authorization processes. Follow these technically oriented, actionable steps designed to enhance security:
  - **Encrypt All Data in Transit:**
  Deploy SSL/TLS encryption protocols for all data exchanged between clients and the server. This is crucial to secure sensitive information like login credentials and personal data from being intercepted during transmission.
  - **Implement Encryption for Data at Rest:**
  Use strong encryption methods such as AES-256 to encrypt stored data within the module. This measure ensures data security, protecting it from unauthorized access if other security controls fail.
  - **Adopt Secure Session Management Practices:**
  Enhance session security by properly managing session tokens through secure generation, storage, and invalidation techniques. This approach helps prevent session hijacking and maintains secure user sessions.

- **Establish and Enforce Data Handling Policies:**
  Develop and enforce strict policies for handling, storing, and transferring sensitive data. Ensure these policies comply with established security best practices and regulatory requirements to minimize data breach risks.
- **Ensure Compliance with Privacy Regulations:**
  Adhere to applicable privacy laws such as the General Data Protection Regulation (GDPR) or the California Consumer Privacy Act (CCPA). Compliance involves setting up the necessary security controls and maintaining transparency in data processing.
- **Perform Regular Security Audits:**
  Conduct frequent security reviews to identify and mitigate vulnerabilities within the module. Regular updates to encryption protocols and data handling policies should be made to respond to evolving security threats effectively.

Rec2. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Component: Redis Cache

2 Recommended countermeasures

Rec1. Cache isolation and mitigation for side-channel attacks C-CACHE-01 Very high Not tested

- **State:** Recommended
- **Description:**
  Implement cache partitioning or isolation techniques to prevent different processes from sharing the same cache regions, mitigating the risk of side-channel attacks. Use cache flush mechanisms to clear sensitive data from the cache after critical operations, and ensure that cryptographic algorithms are designed to be resistant to timing-based attacks. Regularly update system firmware and microcode to patch known cache vulnerabilities.

Rec2. Mitigation of cache timing attacks C-CACHE-03 Very high Not tested

- **State:** Recommended
- **Description:**
  Implement cache timing attack mitigations such as constant-time cryptographic operations to prevent attackers from inferring sensitive data based on timing differences. Use cache flushing techniques to remove sensitive data from the cache immediately after processing. Regularly patch system vulnerabilities and update microcode to address known timing attack vectors.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

2 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side. Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.

Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Component: DIDRegistry Contract

2 Recommended countermeasures

Rec1. Design secure governance systems for Smart Contract C-SMART-CONTRACT-CNT-04 High Not tested

- **State:** Recommended
- **Description:**
Establishing secure governance systems within smart contracts is vital to guarantee decentralized control and align the interests of developers and users. Nevertheless, crafting these governance mechanisms requires meticulous design to minimize the vulnerabilities associated with potential attacks, particularly those stemming from concentrated voting power. This approach offers recommendations to bolster the security of on-chain governance systems, encompassing the utilization of timelocks and techniques to mitigate the risk of rapid voting power accumulation.
Remediation:
1. Implement Timelocks:
- Introduce timelocks in the governance system to prevent immediate execution of critical actions, such as contract upgrades or parameter changes.
- Specify a waiting period, during which proposed actions are held in a pending state, allowing the community time to review and potentially veto them.
- The duration of the timelock should be carefully chosen, balancing the need for security with the necessity of timely governance decisions.
2. Apply Voting Weight Based on Lockup Duration:
- Assign voting weight to tokens based on the duration for which they have been locked up.
- Tokens that have been locked up for longer periods can have proportionally greater voting power.
- This approach incentivizes long-term commitment and reduces the influence of short-term token holders.
3. Use Historical Voting Power Measurements:
- Measure the voting power of an address based on a historical period (e.g., the voting power from 2-3 blocks in the past) rather than the current block.
- This method makes it harder for attackers to quickly accumulate tokens and influence on-chain votes since they would need to hold tokens consistently over time.
References:
  - Smart contract security

Rec2. Reduce code complexity in Smart Contract C-SMART-CONTRACT-CNT-02 Very high Not tested

- **State:** Recommended
- **Description:**
Complexity is a significant source of vulnerabilities in smart contracts. Simplifying the codebase helps reduce the risk of introducing errors and vulnerabilities, making the contract more secure and easier to audit. Reusing established libraries and keeping functions small and modular are effective strategies for achieving code simplicity.
Remediation:
- Whenever possible, leverage well-established libraries like OpenZeppelin Contracts for common functionality, such as token standards. These libraries have undergone extensive testing and auditing, reducing the likelihood of introducing vulnerabilities when implementing standard features.
- Break down the contract's functionality into small, focused functions. Keep functions simple and focused on performing a single task to enhance readability and maintainability. Avoid creating monolithic contracts that combine numerous unrelated functions. Modular contracts make it easier to reason about code correctness and facilitate easier auditing.
References:
  - Smart contract security

Component: RBACManager+Auditlog Contracts

2 Recommended countermeasures

Rec1. Design secure governance systems for Smart Contract C-SMART-CONTRACT-CNT-04 High Not tested

- **State:** Recommended
- **Description:**
Establishing secure governance systems within smart contracts is vital to guarantee decentralized control and align the interests of developers and users. Nevertheless, crafting these governance mechanisms requires meticulous design to minimize the vulnerabilities associated with potential attacks, particularly those stemming from concentrated voting power. This approach offers recommendations to bolster the security of on-chain governance systems, encompassing the utilization of timelocks and techniques to mitigate the risk of rapid voting power accumulation.
Remediation:
1. Implement Timelocks:
- Introduce timelocks in the governance system to prevent immediate execution of critical actions, such as contract upgrades or parameter changes.
- Specify a waiting period, during which proposed actions are held in a pending state, allowing the community time to review and potentially veto them.
- The duration of the timelock should be carefully chosen, balancing the need for security with the necessity of timely governance decisions.
2. Apply Voting Weight Based on Lockup Duration:
- Assign voting weight to tokens based on the duration for which they have been locked up.
- Tokens that have been locked up for longer periods can have proportionally greater voting power.
- This approach incentivizes long-term commitment and reduces the influence of short-term token holders.
3. Use Historical Voting Power Measurements:
- Measure the voting power of an address based on a historical period (e.g., the voting power from 2-3 blocks in the past) rather than the current block.
- This method makes it harder for attackers to quickly accumulate tokens and influence on-chain votes since they would need to hold tokens consistently over time.
References:
  - Smart contract security

Rec2. Reduce code complexity in Smart Contract C-SMART-CONTRACT-CNT-02 Very high Not tested

- **State:** Recommended
- **Description:**
Complexity is a significant source of vulnerabilities in smart contracts. Simplifying the codebase helps reduce the risk of introducing errors and vulnerabilities, making the contract more secure and easier to audit. Reusing established libraries and keeping functions small and modular are effective strategies for achieving code simplicity.
Remediation:
- Whenever possible, leverage well-established libraries like OpenZeppelin Contracts for common functionality, such as token standards. These libraries have undergone extensive testing and auditing, reducing the likelihood of introducing vulnerabilities when implementing standard features.
- Break down the contract's functionality into small, focused functions. Keep functions simple and focused on performing a single task to enhance readability and maintainability. Avoid creating monolithic contracts that combine numerous unrelated functions. Modular contracts make it easier to reason about code correctness and facilitate easier auditing.
References:
  - Smart contract security

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High
Rec2. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

Component: Redis Cache

Rec1. Cache isolation and mitigation for side-channel attacks C-CACHE-01 Very high
Rec2. Mitigation of cache timing attacks C-CACHE-03 Very high

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High
Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

Component: DIDRegistry Contract

Rec1. Design secure governance systems for Smart Contract C-SMART-CONTRACT-CNT-04 High
Rec2. Reduce code complexity in Smart Contract C-SMART-CONTRACT-CNT-02 Very high

Component: RBACManager+Auditlog Contracts

Rec1. Design secure governance systems for Smart Contract C-SMART-CONTRACT-CNT-04 High
Rec2. Reduce code complexity in Smart Contract C-SMART-CONTRACT-CNT-02 Very high

ccpa: Requests to Correct (a)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures (“Imp”) by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended (“Rec”), rejected (“Rej”) or not applicable (“N/A”) in the model, but required by the standard. Required (“ReqF”) countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**

This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Requests to Know (a)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.

4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Requests to Know (b)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(8)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

ccpa: Requirements for Businesses Collecting Large Amounts of Personal Information (a)(1)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:

Enable Auto-Updates for Mobile Apps:
Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
Deploy Vulnerability Scanning Tools:
Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
Integrate with Mobile Device Management (MDM):
Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
Monitor, Report, and Remediate:
Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
References:
- OWASP Mobile Security Testing Guide

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

ccpa: Third Parties (b)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
Implementation Steps:
Enable Auto-Updates for Mobile Apps:
Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
Deploy Vulnerability Scanning Tools:
Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
Integrate with Mobile Device Management (MDM):
Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
Monitor, Report, and Remediate:
Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
References:
- OWASP Mobile Security Testing Guide

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

ccpa: Verification for Non-Accountholders (b)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Verification for Non-Accountholders (c)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Authenticator App(TOTP)

1 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Component: Metaverse user

1 Recommended countermeasures

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High

Component: Metaverse user

Rec1. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Verification for Password-Protected Accounts (a)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Enhance routing security in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-02 Low
2. Implement Sybil Attack prevention measures in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-04 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Mandate high-entropy client secrets and protect against brute-force attacks C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-02 Low

Component: E2E Messaging Relay

Imp1. Implement encryption of communications C-COM-ENCRYPTION Low
Imp2. Implement two-factor authentication to provide an extra layer of security C-2FA Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High Not tested

- **State:** Recommended
- **Description:**
  Implement secure configuration and encryption standards to protect Authentication and Authorization processes. Follow these technically oriented, actionable steps designed to enhance security:
  - **Encrypt All Data in Transit:**
    Deploy SSL/TLS encryption protocols for all data exchanged between clients and the server. This is crucial to secure sensitive information like login credentials and personal data from being intercepted during transmission.
  - **Implement Encryption for Data at Rest:**
    Use strong encryption methods such as AES-256 to encrypt stored data within the module. This measure ensures data security, protecting it from unauthorized access if other security controls fail.
  - **Adopt Secure Session Management Practices:**
    Enhance session security by properly managing session tokens through secure generation, storage, and invalidation techniques. This approach helps prevent session hijacking and maintains secure user sessions.
  - **Establish and Enforce Data Handling Policies:**
    Develop and enforce strict policies for handling, storing, and transferring sensitive data. Ensure these policies comply with established security best practices and regulatory requirements to minimize data breach risks.
  - **Ensure Compliance with Privacy Regulations:**
    Adhere to applicable privacy laws such as the General Data Protection Regulation (GDPR) or the California Consumer Privacy Act (CCPA). Compliance involves setting up the necessary security controls and maintaining transparency in data processing.
  - **Perform Regular Security Audits:**
    Conduct frequent security reviews to identify and mitigate vulnerabilities within the module. Regular updates to encryption protocols and data handling policies should be made to respond to evolving security threats effectively.

Component: RBAC Authorization Engine

1 Recommended countermeasures

Rec1. Enforce HTTPS and modern TLS for secure communication C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-08 High Not tested

- **State:** Recommended
- **Description:**
  Mandate HTTPS for all communication between clients, authorization servers, and resource servers to ensure the confidentiality and integrity of sensitive data. Reject any non-HTTPS requests to prevent exposure to man-in-the-middle (MITM) and eavesdropping attacks. Use TLS 1.2 or higher to ensure robust encryption and protocol security.
  To implement this:

  1. **Enable HTTPS**: Configure the authorization server and resource servers to enforce HTTPS on all endpoints, ensuring all communication is encrypted.
  2. **Disable Non-HTTPS Requests**: Reject any requests made over HTTP or downgrade attempts by redirecting traffic to HTTPS.
  3. **Use Modern TLS**: Configure servers to use TLS 1.2 or higher, and disable outdated protocols like TLS 1.0 and TLS 1.1.
  4. **Update Certificates**: Use valid, up-to-date certificates from a trusted Certificate Authority (CA). Regularly review and renew certificates before expiration.
  5. **Enable HSTS**: Implement HTTP Strict Transport Security (HSTS) headers to prevent protocol downgrades and enforce HTTPS for all subsequent connections.

  For more information, see:
  - RFC 2818: HTTPS

Component: Authenticator App(TOTP)

3 Recommended countermeasures

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update mobile applications along with conducting periodic vulnerability scanning on all mobile devices to ensure that installed apps remain secure and free of known vulnerabilities. This control minimizes the risk of exploitation through outdated or misconfigured applications by automating app updates and integrating vulnerability assessments into the mobile device management process. Developers and DevOps engineers should leverage centralized management tools to enforce update policies and deploy vulnerability scanning solutions to identify and remediate security issues in installed apps.
  Implementation Steps:
  Enable Auto-Updates for Mobile Apps:
  Configure mobile devices and app stores to automatically update applications as new, secure versions are released.
  Deploy Vulnerability Scanning Tools:
  Utilize mobile security tools to conduct regular vulnerability scans on installed applications, identifying potential weaknesses or misconfigurations.
  Integrate with Mobile Device Management (MDM):
  Use MDM solutions to enforce app update policies and schedule regular vulnerability assessments across all devices.
  Monitor, Report, and Remediate:
  Establish processes to monitor scan results, report identified vulnerabilities, and quickly remediate issues through updates or configuration changes.
  References:
  - OWASP Mobile Security Testing Guide

Rec2. Implement strong device authentication C-MOBILE-DEVICE-CLIENT-CNT-01 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update strong device authentication on all mobile devices to ensure that only authorized users can access the device and its sensitive data. This control involves enforcing robust authentication mechanisms—such as multi-factor authentication (MFA), complex passcodes, and biometric verification—to mitigate the risk of unauthorized access. Developers and DevOps engineers should leverage mobile device management (MDM) solutions and enterprise identity providers to enforce and monitor these authentication policies across all devices.
  Implementation Steps:
  Enforce Robust Passcode Policies:
  Configure devices to require complex passcodes with a minimum length and periodic changes.
  Enable Biometric Authentication:
  Utilize built-in biometric features (e.g., fingerprint or facial recognition) to add an additional layer of security.
  Deploy Multi-Factor Authentication (MFA):
  Integrate MFA for accessing sensitive applications and data on mobile devices.
  Integrate with Enterprise Identity Providers:
  Use centralized authentication systems via MDM to enforce consistent policies and monitor compliance across the device fleet.
  Regularly Audit Authentication Settings:
  Conduct periodic reviews and audits to ensure that authentication configurations remain effective and aligned with evolving security requirements.
  References:

- NIST SP 800-63B: Digital Identity Guidelines

Rec3. Use VPN and enforce encrypted communication channels C-MOBILE-DEVICE-CLIENT-CNT-05 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update Virtual Private Network (VPN) configurations and enforce encrypted communication channels on all mobile devices to secure data in transit. This control ensures that all network communications are routed through secure tunnels and that robust encryption protocols (e.g., TLS) are applied, protecting against eavesdropping and man-in-the-middle attacks. Developers and DevOps engineers should leverage mobile device management (MDM) solutions to deploy VPN configurations and enforce encryption standards consistently across the device fleet.
  Implementation Steps:
  Deploy VPN Solutions:
  Configure and enforce the use of reputable VPN solutions on all mobile devices to create secure tunnels for network traffic.
  Enable Strong Encryption Protocols:
  Ensure that both VPN connections and general network communications use the latest approved encryption protocols (e.g., TLS 1.2/1.3).
  Centralized Management:
  Use MDM solutions to enforce VPN usage and encrypted communication policies across all mobile devices, and monitor compliance regularly.
  Test and Audit:
  Conduct periodic security audits and penetration testing to validate that VPN and encrypted channels are correctly implemented and functioning as intended.
  References:
  - NIST Special Publication 800-52 Revision 2

Component: Metaverse user

2 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side. Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High

Component: RBAC Authorization Engine

Rec1. Enforce HTTPS and modern TLS for secure communication C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-08 High

Component: Authenticator App(TOTP)

Rec1. Enforce regular app updates and conduct vulnerability scanning C-MOBILE-DEVICE-CLIENT-CNT-10 High
Rec2. Implement strong device authentication C-MOBILE-DEVICE-CLIENT-CNT-01 High
Rec3. Use VPN and enforce encrypted communication channels C-MOBILE-DEVICE-CLIENT-CNT-05 High

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High
Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Opt-Out Preference Signals (c)(2)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Enhance routing security in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-02 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: E2E Messaging Relay

Imp1. Implement encryption of communications C-COM-ENCRYPTION Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High Not tested

- **State:** Recommended
- **Description:**
  Implement secure configuration and encryption standards to protect Authentication and Authorization processes. Follow these technically oriented, actionable steps designed to enhance security:
  - **Encrypt All Data in Transit:**
    Deploy SSL/TLS encryption protocols for all data exchanged between clients and the server. This is crucial to secure sensitive information like login credentials and personal data from being intercepted during transmission.
  - **Implement Encryption for Data at Rest:**
    Use strong encryption methods such as AES-256 to encrypt stored data within the module. This measure ensures data security, protecting it from unauthorized access if other security controls fail.
  - **Adopt Secure Session Management Practices:**
    Enhance session security by properly managing session tokens through secure generation, storage, and invalidation techniques. This approach helps prevent session hijacking and maintains secure user sessions.
  - **Establish and Enforce Data Handling Policies:**
    Develop and enforce strict policies for handling, storing, and transferring sensitive data. Ensure these policies comply with established security best practices and regulatory requirements to minimize data breach risks.
  - **Ensure Compliance with Privacy Regulations:**
    Adhere to applicable privacy laws such as the General Data Protection Regulation (GDPR) or the California Consumer Privacy Act (CCPA). Compliance involves setting up the necessary security controls and maintaining transparency in data processing.
  - **Perform Regular Security Audits:**
    Conduct frequent security reviews to identify and mitigate vulnerabilities within the module. Regular updates to encryption protocols and data handling policies should be made to respond to evolving security threats effectively.

Component: RBAC Authorization Engine

1 Recommended countermeasures

Rec1. Enforce HTTPS and modern TLS for secure communication C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-08 High Not tested

- **State:** Recommended
- **Description:**
  Mandate HTTPS for all communication between clients, authorization servers, and resource servers to ensure the confidentiality and integrity of sensitive data. Reject any non-HTTPS requests to prevent exposure to man-in-the-middle (MITM) and eavesdropping attacks. Use TLS 1.2 or higher to ensure robust encryption and protocol security.
  To implement this:

  1. **Enable HTTPS**: Configure the authorization server and resource servers to enforce HTTPS on all endpoints, ensuring all communication is encrypted.
  2. **Disable Non-HTTPS Requests**: Reject any requests made over HTTP or downgrade attempts by redirecting traffic to HTTPS.
  3. **Use Modern TLS**: Configure servers to use TLS 1.2 or higher, and disable outdated protocols like TLS 1.0 and TLS 1.1.
  4. **Update Certificates**: Use valid, up-to-date certificates from a trusted Certificate Authority (CA). Regularly review and renew certificates before expiration.
  5. **Enable HSTS**: Implement HTTP Strict Transport Security (HSTS) headers to prevent protocol downgrades and enforce HTTPS for all subsequent connections.

  For more information, see:
  - RFC 2818: HTTPS

Component: Authenticator App(TOTP)

2 Recommended countermeasures

Rec1. Implement strong device authentication C-MOBILE-DEVICE-CLIENT-CNT-01 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update strong device authentication on all mobile devices to ensure that only authorized users can access the device and its sensitive data. This control involves enforcing robust authentication mechanisms—such as multi-factor authentication (MFA), complex passcodes, and biometric verification—to mitigate the risk of unauthorized access. Developers and DevOps engineers should leverage mobile device management (MDM) solutions and enterprise identity providers to enforce and monitor these authentication policies across all devices.
  Implementation Steps:
  Enforce Robust Passcode Policies:
  Configure devices to require complex passcodes with a minimum length and periodic changes.
  Enable Biometric Authentication:
  Utilize built-in biometric features (e.g., fingerprint or facial recognition) to add an additional layer of security.
  Deploy Multi-Factor Authentication (MFA):
  Integrate MFA for accessing sensitive applications and data on mobile devices.
  Integrate with Enterprise Identity Providers:
  Use centralized authentication systems via MDM to enforce consistent policies and monitor compliance across the device fleet.
  Regularly Audit Authentication Settings:
  Conduct periodic reviews and audits to ensure that authentication configurations remain effective and aligned with evolving security requirements.
  References:
  - NIST SP 800-63B: Digital Identity Guidelines

Rec2. Use VPN and enforce encrypted communication channels C-MOBILE-DEVICE-CLIENT-CNT-05 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update Virtual Private Network (VPN) configurations and enforce encrypted communication channels on all mobile devices to secure data in transit. This control ensures that all network communications are routed through secure tunnels and that robust encryption protocols (e.g., TLS) are applied, protecting against eavesdropping and man-in-the-middle attacks. Developers and DevOps engineers should leverage mobile device management (MDM) solutions to deploy VPN configurations and enforce encryption standards consistently across the device fleet.
  Implementation Steps:
  Deploy VPN Solutions:
  Configure and enforce the use of reputable VPN solutions on all mobile devices to create secure tunnels for network traffic.

Enable Strong Encryption Protocols:
Ensure that both VPN connections and general network communications use the latest approved encryption protocols (e.g., TLS 1.2/1.3).
Centralized Management:
Use MDM solutions to enforce VPN usage and encrypted communication policies across all mobile devices, and monitor compliance regularly.
Test and Audit:
Conduct periodic security audits and penetration testing to validate that VPN and encrypted channels are correctly implemented and functioning as intended.
References:
- NIST Special Publication 800-52 Revision 2

Component: Metaverse user

1 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side. Follow these steps:

1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High

Component: RBAC Authorization Engine

Rec1. Enforce HTTPS and modern TLS for secure communication C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-08 High

Component: Authenticator App(TOTP)

Rec1. Implement strong device authentication C-MOBILE-DEVICE-CLIENT-CNT-01 High
Rec2. Use VPN and enforce encrypted communication channels C-MOBILE-DEVICE-CLIENT-CNT-05 High

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High

ccpa: Opt-Out Preference Signals (g)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Enhance routing security in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-02 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: E2E Messaging Relay

Imp1. Implement encryption of communications C-COM-ENCRYPTION Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High Not tested

- **State:** Recommended
- **Description:**
Implement secure configuration and encryption standards to protect Authentication and Authorization processes. Follow these technically oriented, actionable steps designed to enhance security:
  - **Encrypt All Data in Transit:**
  Deploy SSL/TLS encryption protocols for all data exchanged between clients and the server. This is crucial to secure sensitive information like login credentials and personal data from being intercepted during transmission.
  - **Implement Encryption for Data at Rest:**
  Use strong encryption methods such as AES-256 to encrypt stored data within the module. This measure ensures data security, protecting it from unauthorized access if other security controls fail.
  - **Adopt Secure Session Management Practices:**
  Enhance session security by properly managing session tokens through secure generation, storage, and invalidation techniques. This approach helps prevent session hijacking and maintains secure user sessions.

- **Establish and Enforce Data Handling Policies:**
  Develop and enforce strict policies for handling, storing, and transferring sensitive data. Ensure these policies comply with established security best practices and regulatory requirements to minimize data breach risks.
- **Ensure Compliance with Privacy Regulations:**
  Adhere to applicable privacy laws such as the General Data Protection Regulation (GDPR) or the California Consumer Privacy Act (CCPA). Compliance involves setting up the necessary security controls and maintaining transparency in data processing.
- **Perform Regular Security Audits:**
  Conduct frequent security reviews to identify and mitigate vulnerabilities within the module. Regular updates to encryption protocols and data handling policies should be made to respond to evolving security threats effectively.

Component: RBAC Authorization Engine

1 Recommended countermeasures

Rec1. Enforce HTTPS and modern TLS for secure communication C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-08 High Not tested

- **State:** Recommended
- **Description:**
  Mandate HTTPS for all communication between clients, authorization servers, and resource servers to ensure the confidentiality and integrity of sensitive data. Reject any non-HTTPS requests to prevent exposure to man-in-the-middle (MITM) and eavesdropping attacks. Use TLS 1.2 or higher to ensure robust encryption and protocol security.
  To implement this:

  1. **Enable HTTPS**: Configure the authorization server and resource servers to enforce HTTPS on all endpoints, ensuring all communication is encrypted.
  2. **Disable Non-HTTPS Requests**: Reject any requests made over HTTP or downgrade attempts by redirecting traffic to HTTPS.
  3. **Use Modern TLS**: Configure servers to use TLS 1.2 or higher, and disable outdated protocols like TLS 1.0 and TLS 1.1.
  4. **Update Certificates**: Use valid, up-to-date certificates from a trusted Certificate Authority (CA). Regularly review and renew certificates before expiration.
  5. **Enable HSTS**: Implement HTTP Strict Transport Security (HSTS) headers to prevent protocol downgrades and enforce HTTPS for all subsequent connections.

  For more information, see:
  - RFC 2818: HTTPS

Component: Authenticator App(TOTP)

2 Recommended countermeasures

Rec1. Implement strong device authentication C-MOBILE-DEVICE-CLIENT-CNT-01 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update strong device authentication on all mobile devices to ensure that only authorized users can access the device and its sensitive data. This control involves enforcing robust authentication mechanisms—such as multi-factor authentication (MFA), complex passcodes, and biometric verification—to mitigate the risk of unauthorized access. Developers and DevOps engineers should leverage mobile device management (MDM) solutions and enterprise identity providers to enforce and monitor these authentication policies across all devices.
  Implementation Steps:
  Enforce Robust Passcode Policies:
  Configure devices to require complex passcodes with a minimum length and periodic changes.
  Enable Biometric Authentication:
  Utilize built-in biometric features (e.g., fingerprint or facial recognition) to add an additional layer of security.
  Deploy Multi-Factor Authentication (MFA):
  Integrate MFA for accessing sensitive applications and data on mobile devices.
  Integrate with Enterprise Identity Providers:
  Use centralized authentication systems via MDM to enforce consistent policies and monitor compliance across the device fleet.
  Regularly Audit Authentication Settings:
  Conduct periodic reviews and audits to ensure that authentication configurations remain effective and aligned with evolving security requirements.
  References:
  - NIST SP 800-63B: Digital Identity Guidelines

Rec2. Use VPN and enforce encrypted communication channels C-MOBILE-DEVICE-CLIENT-CNT-05 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update Virtual Private Network (VPN) configurations and enforce encrypted communication channels on all mobile devices to secure data in transit. This control ensures that all network communications are routed through secure tunnels and that robust encryption protocols (e.g., TLS) are applied, protecting against eavesdropping and man-in-the-middle attacks. Developers and DevOps engineers should leverage mobile device management (MDM) solutions to deploy VPN configurations and enforce encryption standards consistently across the device fleet.
  Implementation Steps:
  Deploy VPN Solutions:
  Configure and enforce the use of reputable VPN solutions on all mobile devices to create secure tunnels for network traffic.
  Enable Strong Encryption Protocols:
  Ensure that both VPN connections and general network communications use the latest approved encryption protocols (e.g., TLS 1.2/1.3).
  Centralized Management:
  Use MDM solutions to enforce VPN usage and encrypted communication policies across all mobile devices, and monitor compliance regularly.
  Test and Audit:
  Conduct periodic security audits and penetration testing to validate that VPN and encrypted channels are correctly implemented and functioning as intended.
  References:
  - NIST Special Publication 800-52 Revision 2

Component: Metaverse user

1 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side.
  Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High

Component: RBAC Authorization Engine

Rec1. Enforce HTTPS and modern TLS for secure communication C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-08 High

Component: Authenticator App(TOTP)

Rec1. Implement strong device authentication C-MOBILE-DEVICE-CLIENT-CNT-01 High
Rec2. Use VPN and enforce encrypted communication channels C-MOBILE-DEVICE-CLIENT-CNT-05 High

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High

ccpa: Verification for Password-Protected Accounts (b)

NotApplicable countermeasures

Component: Sepolia Blockchain+infura

1. Enhance routing security in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-02 Low

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: E2E Messaging Relay

Imp1. Implement encryption of communications C-COM-ENCRYPTION Low

Component: Metaverse user

Imp1. Consider defenses against adversarial tampering such as adversarial training methods or data preprocessing-based approaches C-ML-AI-ADVERSARIAL-ATTACKS High
Imp2. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High Not tested

- **State:** Recommended
- **Description:**
  Implement secure configuration and encryption standards to protect Authentication and Authorization processes. Follow these technically oriented, actionable steps designed to enhance security:
  - **Encrypt All Data in Transit:**
    Deploy SSL/TLS encryption protocols for all data exchanged between clients and the server. This is crucial to secure sensitive information like login credentials and personal data from being intercepted during transmission.
  - **Implement Encryption for Data at Rest:**
    Use strong encryption methods such as AES-256 to encrypt stored data within the module. This measure ensures data security, protecting it from unauthorized access if other security controls fail.
  - **Adopt Secure Session Management Practices:**
    Enhance session security by properly managing session tokens through secure generation, storage, and invalidation techniques. This approach helps prevent session hijacking and maintains secure user sessions.
  - **Establish and Enforce Data Handling Policies:**
    Develop and enforce strict policies for handling, storing, and transferring sensitive data. Ensure these policies comply with established security best practices and regulatory requirements to minimize data breach risks.
  - **Ensure Compliance with Privacy Regulations:**
    Adhere to applicable privacy laws such as the General Data Protection Regulation (GDPR) or the California Consumer Privacy Act (CCPA). Compliance involves setting up the necessary security controls and maintaining transparency in data processing.
  - **Perform Regular Security Audits:**
    Conduct frequent security reviews to identify and mitigate vulnerabilities within the module. Regular updates to encryption protocols and data handling policies should be made to respond to evolving security threats effectively.

Component: RBAC Authorization Engine

1 Recommended countermeasures

Rec1. Enforce HTTPS and modern TLS for secure communication C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-08 High Not tested

- **State:** Recommended
- **Description:**
  Mandate HTTPS for all communication between clients, authorization servers, and resource servers to ensure the confidentiality and integrity of sensitive data. Reject any non-HTTPS requests to prevent exposure to man-in-the-middle (MITM) and eavesdropping attacks. Use TLS 1.2 or higher to ensure robust encryption and protocol security.
  To implement this:

  1. **Enable HTTPS**: Configure the authorization server and resource servers to enforce HTTPS on all endpoints, ensuring all communication is encrypted.
  2. **Disable Non-HTTPS Requests**: Reject any requests made over HTTP or downgrade attempts by redirecting traffic to HTTPS.
  3. **Use Modern TLS**: Configure servers to use TLS 1.2 or higher, and disable outdated protocols like TLS 1.0 and TLS 1.1.
  4. **Update Certificates**: Use valid, up-to-date certificates from a trusted Certificate Authority (CA). Regularly review and renew certificates before expiration.
  5. **Enable HSTS**: Implement HTTP Strict Transport Security (HSTS) headers to prevent protocol downgrades and enforce HTTPS for all subsequent connections.

  For more information, see:

- RFC 2818: HTTPS

Component: Authenticator App(TOTP)

2 Recommended countermeasures

Rec1. Implement strong device authentication C-MOBILE-DEVICE-CLIENT-CNT-01 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update strong device authentication on all mobile devices to ensure that only authorized users can access the device and its sensitive data. This control involves enforcing robust authentication mechanisms—such as multi-factor authentication (MFA), complex passcodes, and biometric verification—to mitigate the risk of unauthorized access. Developers and DevOps engineers should leverage mobile device management (MDM) solutions and enterprise identity providers to enforce and monitor these authentication policies across all devices.
  Implementation Steps:
  Enforce Robust Passcode Policies:
  Configure devices to require complex passcodes with a minimum length and periodic changes.
  Enable Biometric Authentication:
  Utilize built-in biometric features (e.g., fingerprint or facial recognition) to add an additional layer of security.
  Deploy Multi-Factor Authentication (MFA):
  Integrate MFA for accessing sensitive applications and data on mobile devices.
  Integrate with Enterprise Identity Providers:
  Use centralized authentication systems via MDM to enforce consistent policies and monitor compliance across the device fleet.
  Regularly Audit Authentication Settings:
  Conduct periodic reviews and audits to ensure that authentication configurations remain effective and aligned with evolving security requirements.
  References:
  - NIST SP 800-63B: Digital Identity Guidelines

Rec2. Use VPN and enforce encrypted communication channels C-MOBILE-DEVICE-CLIENT-CNT-05 High Not tested

- **State:** Recommended
- **Description:**
  Implement and regularly update Virtual Private Network (VPN) configurations and enforce encrypted communication channels on all mobile devices to secure data in transit. This control ensures that all network communications are routed through secure tunnels and that robust encryption protocols (e.g., TLS) are applied, protecting against eavesdropping and man-in-the-middle attacks. Developers and DevOps engineers should leverage mobile device management (MDM) solutions to deploy VPN configurations and enforce encryption standards consistently across the device fleet.
  Implementation Steps:
  Deploy VPN Solutions:
  Configure and enforce the use of reputable VPN solutions on all mobile devices to create secure tunnels for network traffic.
  Enable Strong Encryption Protocols:
  Ensure that both VPN connections and general network communications use the latest approved encryption protocols (e.g., TLS 1.2/1.3).
  Centralized Management:
  Use MDM solutions to enforce VPN usage and encrypted communication policies across all mobile devices, and monitor compliance regularly.
  Test and Audit:
  Conduct periodic security audits and penetration testing to validate that VPN and encrypted channels are correctly implemented and functioning as intended.
  References:
  - NIST Special Publication 800-52 Revision 2

Component: Metaverse user

2 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side. Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High Not tested

- **State:** Recommended
- **Description:**
  This can be considered as part of the data preprocessing functions already mentioned in the overall defenses against adversarial tampering.
  Anomaly detection, for instance, can be a powerful defense mechanism in this case. It would allow to model the distribution of the clean data, i.e., expected values or patterns, and consequently allows to (potentially) reject anomalous inputs that are likely to be malicious. Consider. when possible, to:

  1. Develop algorithms to model the distribution of clean data, capturing expected values and patterns.
  2. Utilize anomaly detection techniques to differentiate between normal and anomalous inputs.
  3. Configure the system to reject inputs that deviate significantly from the expected distribution, potentially indicating malicious intent.
  4. Regularly update and refine the anomaly detection models based on new data and emerging threats.
  5. Follow best practices for anomaly detection, such as setting appropriate thresholds and monitoring for false positives/negatives.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Enforce secure configuration and encryption C-SECURITY-MEASURES-FOR-CONFIGURATION-AND-COMMUNICATIONS High

Component: RBAC Authorization Engine

Rec1. Enforce HTTPS and modern TLS for secure communication C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-08 High

Component: Authenticator App(TOTP)

Rec1. Implement strong device authentication C-MOBILE-DEVICE-CLIENT-CNT-01 High
Rec2. Use VPN and enforce encrypted communication channels C-MOBILE-DEVICE-CLIENT-CNT-05 High

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High
Rec2. Perform input validation, e.g., via anomaly detection C-ML-AI-INPUT-VALIDATION High

ccpa: Requests to Know (k)(1)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: DIDRegistry Contract

Imp1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: RBACManager+Auditlog Contracts

1 Recommended countermeasures

Rec1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 High Not tested

- **State:** Recommended
- **Description:**
  Ensuring the security and correctness of smart contracts is paramount due to their immutability once deployed. Rigorous testing and verification processes are essential to identify and rectify vulnerabilities, protecting users and assets from potential risks. Combining various testing techniques, including unit testing, property-based testing, static analysis, dynamic analysis (such as fuzzing), and formal verification, enhances the overall security of smart contracts.
  Remediation:
  1. Unit Testing
  - Write small unit tests using mock data that simulates user interactions and contract inputs.
  - Use unit tests to verify the functionality of specific functions within the smart contract.
  - Ensure unit tests cover a wide range of scenarios, including edge cases.
  2. Property-Based Testing
  - Combine unit testing with property-based testing techniques for comprehensive security assessment.
  - Use static analysis to analyze control flow graphs and abstract syntax trees to identify potential vulnerabilities.
  - Apply dynamic analysis techniques like fuzzing to execute contract code with random input values and detect security violations.
  3. Formal Verification:
  - Implement formal verification to conclusively prove the absence of errors in the smart contract.
  - Create a formal specification that captures desired security properties.
  - Prove that the formal model of the contract adheres to the specification.
  References:
  - Smart contract security

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: RBACManager+Auditlog Contracts

Rec1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 High

ccpa: Restrictions on the Collection and Use of Personal Information (c)(2)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: DIDRegistry Contract

Imp1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: RBACManager+Auditlog Contracts

1 Recommended countermeasures

Rec1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 High Not tested

- **State:** Recommended
- **Description:**
  Ensuring the security and correctness of smart contracts is paramount due to their immutability once deployed. Rigorous testing and verification processes are essential to identify and rectify vulnerabilities, protecting users and assets from potential risks. Combining various testing techniques, including unit testing, property-based testing, static analysis, dynamic analysis (such as fuzzing), and formal verification, enhances the overall security of smart contracts.
  Remediation:
  1. Unit Testing
  - Write small unit tests using mock data that simulates user interactions and contract inputs.
  - Use unit tests to verify the functionality of specific functions within the smart contract.
  - Ensure unit tests cover a wide range of scenarios, including edge cases.

2. Property-Based Testing
- Combine unit testing with property-based testing techniques for comprehensive security assessment.
- Use static analysis to analyze control flow graphs and abstract syntax trees to identify potential vulnerabilities.
- Apply dynamic analysis techniques like fuzzing to execute contract code with random input values and detect security violations.
3. Formal Verification:
- Implement formal verification to conclusively prove the absence of errors in the smart contract.
- Create a formal specification that captures desired security properties.
- Prove that the formal model of the contract adheres to the specification.
References:
- Smart contract security

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: RBACManager+Auditlog Contracts

Rec1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 High

ccpa: Restrictions on the Collection and Use of Personal Information (f)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: DIDRegistry Contract

Imp1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: RBACManager+Auditlog Contracts

1 Recommended countermeasures

Rec1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 High Not tested

- **State:** Recommended
- **Description:**
Ensuring the security and correctness of smart contracts is paramount due to their immutability once deployed. Rigorous testing and verification processes are essential to identify and rectify vulnerabilities, protecting users and assets from potential risks. Combining various testing techniques, including unit testing, property-based testing, static analysis, dynamic analysis (such as fuzzing), and formal verification, enhances the overall security of smart contracts.
Remediation:
1. Unit Testing
- Write small unit tests using mock data that simulates user interactions and contract inputs.
- Use unit tests to verify the functionality of specific functions within the smart contract.
- Ensure unit tests cover a wide range of scenarios, including edge cases.
2. Property-Based Testing
- Combine unit testing with property-based testing techniques for comprehensive security assessment.
- Use static analysis to analyze control flow graphs and abstract syntax trees to identify potential vulnerabilities.
- Apply dynamic analysis techniques like fuzzing to execute contract code with random input values and detect security violations.
3. Formal Verification:
- Implement formal verification to conclusively prove the absence of errors in the smart contract.
- Create a formal specification that captures desired security properties.
- Prove that the formal model of the contract adheres to the specification.
References:
- Smart contract security

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: RBACManager+Auditlog Contracts

Rec1. Test and verify Smart Contracts for code correctness C-SMART-CONTRACT-CNT-06 High

ccpa: Record-Keeping (a)

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Create a policy and workflow for comprehensive logging and monitoring C-COMPREHENSIVE-LOGGING-AND-MONITORING Medium Not tested

- **State:** Recommended
- **Description:**
Establish a detailed logging and monitoring strategy to better secure your Authentication and Authorization processes. This strategy is crucial for identifying and responding to security threats promptly. Follow these steps to implement an effective logging and monitoring system:

- **Implement Logging:** Ensure every access attempt, both successful and failed, is logged. Record key details such as timestamps, source IP addresses, user identifiers, and the specifics of the access attempt.
- **Log Sensitive User Actions:** Track and store logs of critical user activities, particularly those involving sensitive data or administrative functions. These logs are vital for forensic analysis and auditing purposes.
- **Set Up Real-time Monitoring:** Deploy real-time monitoring tools that continuously analyze log data. Configure these tools to recognize patterns indicative of potential security breaches or anomalies.
- **Establish Automated Alerts:** Create alerting mechanisms that notify system administrators immediately when suspicious activities are detected. Tailor the alert thresholds to minimize false positives while ensuring prompt response to real threats.
- **Conduct Regular Log Reviews:** Schedule periodic log reviews by trained security personnel to spot trends or issues that automated systems might overlook. This practice helps in proactive threat detection and enhances overall security vigilance.

By embedding these logging and monitoring practices into your security protocols, you can significantly improve your ability to detect, investigate, and mitigate security incidents efficiently. For best practices, refer to the NIST guidelines on log management and the ISO/IEC standards on information security monitoring.

Component: RBAC Authorization Engine

1 Recommended countermeasures

Rec1. Log key actions and monitor for unusual activity C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-07 Medium Not tested

- **State:** Recommended
- **Description:**
  Implement comprehensive logging for critical events, such as token issuance, authentication attempts, and authorization requests, to enhance the visibility of potential threats to the OAuth2 Authorization Server. Real-time monitoring and alerts should be established to detect and respond to suspicious activity promptly.
  To implement this:

  1. **Log Critical Events**: Record all significant actions, including token issuance, token refresh, failed authentication attempts, token revocation, and authorization errors, with timestamps, client identifiers, and IP addresses.
  2. **Ensure Secure Log Storage**: Protect logs from tampering and unauthorized access by encrypting them and using secure storage solutions.
  3. **Set Up Monitoring Tools**: Use SIEM (Security Information and Event Management) tools to analyze logs in real time and detect anomalies, such as an unusually high rate of token requests from a single client.
  4. **Configure Alerts**: Implement automatic alerts for patterns of suspicious behavior, such as repeated failed authentication attempts or unusual geolocation changes.

  For more information, see:
  - OWASP Logging Cheat Sheet

Component: Metaverse user

1 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side.
  Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Create a policy and workflow for comprehensive logging and monitoring C-COMPREHENSIVE-LOGGING-AND-MONITORING Medium

Component: RBAC Authorization Engine

Rec1. Log key actions and monitor for unusual activity C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-07 Medium

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High

ccpa: Methods for Submitting Requests to Delete Requests to Correct and Requests to Know (c)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- State: Recommended
- Description:
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Methods for Submitting Requests to Delete Requests to Correct and Requests to Know (d)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- State: Recommended
- Description:
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Notice at Collection of Personal Information (a)(b)(c)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Notice at Collection of Personal Information (g)(1)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Notice at Collection of Personal Information (g)(2)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Opt-Out Preference Signals (c)(6)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended

- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Opt-Out Preference Signals (d)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Privacy Policy (e)(2)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Privacy Policy (e)(3)(E)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Privacy Policy (e)(3)(G)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Privacy Policy (e)(1)(I)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.

- **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
- **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
- **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
- **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Correct (k)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- State: Recommended
- Description:
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Delete (b)(3)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (a)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Opt-Out of Sale/Sharing (a)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- State: Recommended
- Description:
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Opt-Out of Sale/Sharing (d)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- State: Recommended
- Description:
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Opt-Out of Sale/Sharing (f)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Opt-Out of Sale/Sharing (g)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Opt-Out of Sale/Sharing (h)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Opt-Out of Sale/Sharing (j)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Requests to Opt-Out of Sale/Sharing (k)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Restrictions on the Collection and Use of Personal Information (b)(5)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Restrictions on the Collection and Use of Personal Information (c)(3)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- **State:** Recommended
- **Description:**
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:

- **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
- **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
- **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
- **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
- **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
- **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
- **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Service Providers and Contractors (a)(5)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Bind authorization codes to specific clients and enforce PKCE for public clients C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-04 Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Fast API Auth Service

1 Recommended countermeasures

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High Not tested

- State: Recommended
- Description:
  Implement rate limiting and resource management to protect against enumeration and denial of service (DoS) attacks. These practices help control the flow of requests, preventing system overload and ensuring service availability for legitimate users.
  Implementation Steps:
  - **Generic Error Messages:** Always return generic error messages for failed login attempts or user queries. For example, use messages like 'Invalid username or password' instead of specifying which part of the input was incorrect.
  - **Identify Key Resources:** Assess which parts of your authentication system are most vulnerable to high traffic and focus your rate limiting efforts there.
  - **Establish Request Thresholds:** Set practical rate limits based on typical user behavior. Apply stricter limits for sensitive actions and more lenient ones for general requests.
  - **Select a Rate Limiting Method:** Choose an appropriate rate limiting mechanism such as the Leaky Bucket or Token Bucket algorithm, depending on your system's architecture and traffic pattern.
  - **Implement the Rate Limiter:** Integrate rate limiting directly into your application logic or use existing middleware solutions that support rate limiting, ensuring it aligns with your system's scalability requirements.
  - **Manage System Resources:** Continuously monitor and adjust resource allocation, using techniques like load balancing and connection pooling to handle traffic efficiently.
  - **Inform Users:** Provide feedback to users about their request status, using HTTP 429 'Too Many Requests' responses when limits are exceeded.

  By adopting these steps, you enhance the resilience of your Authentication and Authorization Module against disruptive traffic patterns while maintaining a smooth and secure user experience. Refer to the OWASP Secure Coding Practices and NIST guidelines on web application security for further details and best practices in implementing these controls.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Fast API Auth Service

Rec1. Rate limiting and proper resource management C-RATE-LIMITING-AND-RESOURCE-MANAGEMENT High

ccpa: Opt-Out Preference Signals (c)(5)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: PostgreSQL Database

Imp1. Secure setup and access control C-DBAAS-DATABASE-AS-A-SERVICE-SECURE-SETUP Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Metaverse user

1 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side.
  Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High

ccpa: Record-Keeping (d)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: RBAC Authorization Engine

Imp1. Enforce principle of least privilege in scope management C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-06 Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Metaverse user

1 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side.
  Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(2)

Implemented countermeasures

Below are the implemented countermeasures ("Imp") by component and threat for standard reference.

Component: PostgreSQL Database

Imp1. Secure setup and access control C-DBAAS-DATABASE-AS-A-SERVICE-SECURE-SETUP Low

Component: Metaverse user

Imp1. Protect the user's data and interaction, and data streams used in prediction C-ML-AI-PROTECT-DATA-STREAMS Low

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Metaverse user

1 Recommended countermeasures

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High Not tested

- **State:** Recommended
- **Description:**
  Developers should always ensure rigorous controls are implemented at all levels to regulate access to the model, and its data or components (if applicable), including on the user/usage side.
  Follow these steps:

  1. Restrict access to the model and data by implementing secure authentication mechanisms and role-based access controls when applicable.
  2. Regularly review and update access permissions to ensure only authorized personnel can interact with the model and its components.
  3. Encrypt data at rest and in transit to safeguard it from unauthorized access.

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Metaverse user

Rec1. Control users' access to models C-ML-AI-ACCESS-CONTROL-USER High

ccpa: Record-Keeping (b)

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: RBAC Authorization Engine

1 Recommended countermeasures

Rec1. Log key actions and monitor for unusual activity C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-07 Medium Not tested

- **State:** Recommended
- **Description:**
  Implement comprehensive logging for critical events, such as token issuance, authentication attempts, and authorization requests, to enhance the visibility of potential threats to the OAuth2 Authorization Server. Real-time monitoring and alerts should be established to detect and respond to suspicious activity promptly.
  To implement this:

  1. **Log Critical Events**: Record all significant actions, including token issuance, token refresh, failed authentication attempts, token revocation, and authorization errors, with timestamps, client identifiers, and IP addresses.
  2. **Ensure Secure Log Storage**: Protect logs from tampering and unauthorized access by encrypting them and using secure storage solutions.
  3. **Set Up Monitoring Tools**: Use SIEM (Security Information and Event Management) tools to analyze logs in real time and detect anomalies, such as an unusually high rate of token requests from a single client.
  4. **Configure Alerts**: Implement automatic alerts for patterns of suspicious behavior, such as repeated failed authentication attempts or unusual geolocation changes.

  For more information, see:
  - OWASP Logging Cheat Sheet

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: RBAC Authorization Engine

Rec1. Log key actions and monitor for unusual activity C-OAUTH2-AUTHORIZATION-SERVER-CNT-OAUTH2-AS-07 Medium

ccpa: Requests to Limit Use and Disclosure of Sensitive Personal Information (m)(3)

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Sepolia Blockchain+infura

1 Recommended countermeasures

Rec1. Implement effective endpoint security in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-01 Medium Not tested

- **State:** Recommended
- **Description:**
  To ensure the security and integrity of blockchain nodes within a network, deploying a robust endpoint protection solution is crucial. Follow these step-by-step guidelines to implement this countermeasure effectively:

  1. **Assess the Current Environment:**
     - Perform a comprehensive assessment of the existing blockchain node infrastructure to identify potential vulnerabilities and entry points for unauthorized access.
     - Document the types of operating systems in use, as endpoint protection solutions can vary based on the OS compatibility.
  2. **Select an Endpoint Protection Solution:**
     - Research and choose an endpoint protection solution that offers real-time monitoring, threat detection, and response capabilities tailored for blockchain environments.
     - Ensure the solution supports intrusion detection, malware protection, and encryption techniques.
     - Consider solutions with centralized management features for ease of deployment and monitoring across multiple nodes.

3. **Deploy the Selected Solution:**
   - Install the endpoint protection software on each blockchain node, applying consistent configuration settings as per your security policy.
   - Set up real-time monitoring and alert systems to notify the security team about suspicious activities or access attempts.
4. **Configure Access Controls:**
   - Implement strict access control policies to ensure only authorized personnel have access to the nodes.
   - Utilize multi-factor authentication for accessing node management interfaces and data.
5. **Regular Updates and Patches:**
   - Establish a schedule for regular updates to the endpoint protection software to ensure it can combat the latest threats effectively.
   - Regularly update the blockchain software and apply security patches promptly to mitigate known vulnerabilities.
6. **Continuous Monitoring and Improvement:**
   - Conduct regular security audits and penetration testing to assess the effectiveness of the endpoint protection solution.
   - Analyze audit logs and review incident responses for continuous enhancement of security policies and practices.
7. **Training and Awareness:**
   - Provide regular training sessions for employees and contractors on security best practices specific to maintaining blockchain node integrity.
   - Educate users on identifying potential threats, such as phishing campaigns targeting blockchain credentials.

References

- Protocols and Guidelines to Enhance the Endpoint Security of Blockchain at User's End
- Common blockchain security issues and how to prevent them

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Sepolia Blockchain+infura

Rec1. Implement effective endpoint security in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-01 Medium

ccpa: Sworn Complaints Filed with the Agency (a)(b)

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Sepolia Blockchain+infura

1 Recommended countermeasures

Rec1. Implement effective endpoint security in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-01 Medium Not tested

- **State:** Recommended
- **Description:**
  To ensure the security and integrity of blockchain nodes within a network, deploying a robust endpoint protection solution is crucial. Follow these step-by-step guidelines to implement this countermeasure effectively:

  1. **Assess the Current Environment:**
     - Perform a comprehensive assessment of the existing blockchain node infrastructure to identify potential vulnerabilities and entry points for unauthorized access.
     - Document the types of operating systems in use, as endpoint protection solutions can vary based on the OS compatibility.
  2. **Select an Endpoint Protection Solution:**
     - Research and choose an endpoint protection solution that offers real-time monitoring, threat detection, and response capabilities tailored for blockchain environments.
     - Ensure the solution supports intrusion detection, malware protection, and encryption techniques.
     - Consider solutions with centralized management features for ease of deployment and monitoring across multiple nodes.
  3. **Deploy the Selected Solution:**
     - Install the endpoint protection software on each blockchain node, applying consistent configuration settings as per your security policy.
     - Set up real-time monitoring and alert systems to notify the security team about suspicious activities or access attempts.
  4. **Configure Access Controls:**
     - Implement strict access control policies to ensure only authorized personnel have access to the nodes.
     - Utilize multi-factor authentication for accessing node management interfaces and data.
  5. **Regular Updates and Patches:**
     - Establish a schedule for regular updates to the endpoint protection software to ensure it can combat the latest threats effectively.
     - Regularly update the blockchain software and apply security patches promptly to mitigate known vulnerabilities.
  6. **Continuous Monitoring and Improvement:**
     - Conduct regular security audits and penetration testing to assess the effectiveness of the endpoint protection solution.
     - Analyze audit logs and review incident responses for continuous enhancement of security policies and practices.
  7. **Training and Awareness:**
     - Provide regular training sessions for employees and contractors on security best practices specific to maintaining blockchain node integrity.
     - Educate users on identifying potential threats, such as phishing campaigns targeting blockchain credentials.

References

- Protocols and Guidelines to Enhance the Endpoint Security of Blockchain at User's End
- Common blockchain security issues and how to prevent them

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Sepolia Blockchain+infura

Rec1. Implement effective endpoint security in Blockchain Network C-BLOCKCHAIN-NETWORK-CNT-01 Medium

End of Compliance report